

Deep Learning Assignment Report

Adam Hrehovcik

1 Introduction

This report presents the design, implementation, and evaluation of statistical and deep learning language models trained on the IMDb ACL movie review dataset. The objective of the project was threefold. First, to construct a transparent and interpretable trigram language model as a statistical baseline. Second, to implement a recurrent neural network language model using a reduced vocabulary to ensure feasible computational time. Third, to evaluate generative quality both intrinsically through perplexity metric and externally through a discriminative classifier trained to distinguish real from generated sentences.

The modelling process emphasised deliberate control of design choices and overall methodology. Rather than relying on prebuilt tokenisation pipelines or pretrained language models, the pipeline was constructed from first principles to ensure interpretability and analytical control over the key decision variables.

2 Part 1: Tokenization

2.1 Tokenization Strategy

Once the data was loaded from positive and negative sentiment review files, with sentiment labels ignored since the task focuses on pure language modelling, a simple and transparent tokenizer was built and implemented, as detailed in the Code and Pipeline Notebook.

A word-level tokenization was chosen because of its high degree of interpretability and computational simplicity, and flexibility. First, as each token corresponds to a human-understandable word, inspecting the tokenizer processes at different stages is quite simple, and so is qualitative inspection of the model outputs. Second, punctuation removal and whitespace splitting are simple and computationally efficient and produce a clean vocabulary that is directly suitable for standard N-gram modelling. Third, this approach offers inherent flexibility in vocabulary manipulation, as using human-interpretable words makes it straightforward to inspect, filter, and modify the vocabulary based on corpus statistics. In fact, the word-level design became particularly advantageous when later reducing vocabulary for neural network training, as frequency-based pruning naturally retains common words while removing the rare, potentially less useful words.

However, this choice also introduces limitations. Word-level tokenisation leads to a relatively large vocabulary compared to subword or character approaches. This becomes especially important when training language models, since larger vocabulary means larger embedding matrices and a larger output layer (the softmax must compute probabilities over every word in the vocabulary), both of which increase memory usage and slow down training and inference. Word-level tokenization also cannot deal with out-of-vocabulary words beyond mapping them as "unknown" token, losing all the information about structure of the word in the process. Subword methods such as BPE, by contrast, can decompose an unseen word into familiar subword units, preserving meaningful components such as roots, prefixes, and suffixes. In addition, the removal of punctuation and more advanced linguistic markers discards potentially informative syntactic information.

These trade-offs were considered acceptable because the objective was probabilistic sequence modelling rather than detailed syntactic and semantic analysis.

Additionally, special tokens were introduced to support modelling objectives. The `<START>` and `<END>` tokens explicitly mark sentence boundaries, enabling both the trigram and RNN models to learn when sequences begin and terminate. The `<PAD>` token was introduced to support fixed-length sequence batching used for RNN training later on. The `<UNK>` token provides a controlled mechanism for handling rare or unseen words after the vocabulary reduction.

3 Part 2: Non-deep Learning Model (N-gram)

3.1 Model Specification Rationale

A trigram language model ($N=3$) was implemented as a statistical baseline. The model estimates conditional probabilities of the form:

$$P(w_3|w_1, w_2) \tag{1}$$

where the probability of the next token depends only on the two preceding tokens.

The choice of $N = 3$ reflects a deliberate bias-variance tradeoff. A bigram model ($N = 2$) often produces overly generic and contextually weak predictions because conditioning on a single previous token does not capture wider context in which words are used or syntactic dependencies. Conversely, a four-gram model ($N = 4$) would theoretically capture richer context but introduces serious sparsity problems given limited sample size. The exponential growth in possible four-word contexts given a fixed vocabulary size means that most contexts observed during generation would not have appeared during training. As a result, the model would spend the vast majority of its probability estimates relying on smoothing rather than actual observed counts, worsening its ability to make meaningful probability estimates.

The trigram model therefore represents a pragmatic middle ground for this task. It is expressive enough to capture common short phrases in movie reviews (e.g., “one of the”, “I did not”, “this movie is”), yet sufficiently data-efficient to produce stable probability estimates.

3.2 Smoothing Strategy

Even with trigrams, many context-continuation combinations remain unseen. Smoothing is therefore necessary, particularly for perplexity evaluation, where zero probabilities would make the metric undefined.

Laplace smoothing with parameter $k=1$ was applied:

$$P(w_3|w_1, w_2) = \frac{c(w_1, w_2, w_3) + 1}{\sum_{w'} c(w_1, w_2, w') + |V|} \tag{2}$$

This strategy assigns a count of one, and thus by effect a small non-zero probability, to unseen events while retaining higher relative mass for true observed trigrams.

Laplace smoothing offers two primary advantages. First, it is simple and computationally inexpensive, requiring only adjusted counts. Second, since every possible trigram is assigned a non-zero probability, perplexity is guaranteed to be well-defined and finite (perplexity takes the log of these probabilities), without needing to tune an additional hyperparameter beyond the choice of $k=1$.

However, Laplace smoothing has also notable disadvantages. It redistributes probability mass uniformly across the entire vocabulary, including many implausible continuations. This effectively subtracts probability from high-confidence frequent events and reallocates it to unlikely ones. As vocabulary size increases, the total amount of probability mass redistributed grows meaning the estimates for well-observed, frequent trigrams are discounted more heavily.

Despite these disadvantages, Laplace smoothing was chosen since the objective was to obtain a stable, interpretable baseline model with guaranteed non-zero probabilities without the need for excessive sophistication.

In this specific context, the choice of $k=1$ reflects a conservative approach compared to where unseen continuations receive a limited probability mass, compared to higher values of k . It should be noted that smoothing’s impact is relative to the vocabulary size, so in this case, given the vocabulary size of over 100,000, the $k=1$ spreads probability mass relatively thinly across unseen events. This ensures that generation remains grounded in frequently observed review patterns. Given that movie reviews are relatively narrow in stylistic scope and often reuse common constructions, preserving observed frequency structure and sampling from observed trigrams where possible is desirable in this case.

3.3 Sampling Strategies

Two decoding strategies were implemented for text generation:

- **Greedy decoding:** Selects the most probable next token at each step.
- **Top- k sampling ($k=10$):** Samples from the ten most probable tokens and renormalizes their probabilities to form a new complete probability distribution.

Greedy decoding maximises local likelihood and typically produces syntactically plausible continuations. However, because the model is deterministic under this strategy, it repeatedly generates the same highest-probability sequence given identical starting conditions. This reduces diversity and can lead to repetitive phrasing.

Top- k sampling introduces controlled stochasticity. By restricting sampling to the ten most probable continuations, the method avoids extremely low-probability tokens that would generate very unlikely and often nonsensical output, while still allowing variability across runs. This provides a compromise between strict likelihood maximisation and complete random sampling, which is more prone to illogical next-token generation.

It should be noted that $k=10$ is a fixed and somewhat arbitrary choice. In practice, the optimal candidate set size varies depending on the probability distribution at each step. For instance, if the distribution is highly peaked, a smaller k may be sufficient, whereas a flatter distribution might warrant a larger k . This fixed nature means the method cannot adapt to the varying degrees of certainty the model has at different points in generation, which is a fundamental limitation of the approach.

A shared limitation of both approaches is that decoding decisions are made purely locally. This means that each token is selected based only on the immediate trigram context, with no way to ensure coherence with wider context is maintained.

3.4 Empirical Behaviour and Limitations

Observing the generated samples reveals several patterns.

Method	Generated Text
Greedy	the film is a very good
Top-10 (Sample 1)	the film was made in the end
Top-10 (Sample 2)	the acting was awful
Top-10 (Sample 3)	and i was watching a very bad acting and a half of the film is that it was very good
Top-10 (Sample 4)	i dont see this movie is just an example of a story that it was
Top-10 (Sample 5)	this film

Table 1: Comparison of Greedy and Top-k Sampling Outputs | Trigram

First, trigram generation is locally coherent. Short phrases and immediate continuations are often grammatically sound and stylistically aligned with movie reviews. This confirms that the model captures short-range dependencies effectively.

Second, long-term coherence is poor. Because the trigram model conditions only on the two most recent tokens, it lacks memory of earlier thematic content. As a result, generated sentences may begin plausibly but drift into illogical word sequences that lack any coherent narrative.

Third, sentence endings are frequent and abrupt. The `<END>` token is generated whenever the local trigram context suggests a plausible stopping point, and since it has no awareness of how much text has been generated or whether a coherent complete sentence has been formed, it tends to terminate prematurely as soon as the local likelihood of continuing declines.

Overall, the trigram model serves as a strong interpretable baseline. It captures local word structure but is fundamentally limited by its limited context window. These limitations motivate the transition to recurrent neural architectures capable of modelling longer-range dependencies.

4 Part 3: Deep Learning Model

4.1 Vocabulary Reduction

To ensure computational feasibility of the neural language model, a reduced vocabulary was constructed by retaining only the top 20% of most frequent tokens from the training corpus, while always preserving the special tokens `<PAD>`, `<UNK>`, `<START>`, and `<END>`. All other tokens were mapped to `<UNK>`.

Metric	Value
Reduced Vocabulary Size	20,823
Top Non-Special Tokens Retained	20,819
Original Non-Special Tokens	104,096

Table 2: Vocabulary Reduction Summary

The primary motivation for the vocabulary reduction was computational. In a standard LSTM language model, the softmax output layer projects the hidden state to a probability distribution over the entire vocabulary. Hence, the computational cost of this projection is proportional to the vocabulary size. For large vocabularies, the softmax layer often becomes the dominant cost in both memory usage and training time. Reducing the vocabulary therefore directly reduces the number of dimensions of the output layer and substantially lowers training complexity.

However, the benefits of vocabulary reduction extend beyond computational efficiency. A smaller vocabulary reduces the total number of embedding parameters that must be learned. Additionally, many rare words appear only a

couple of times in the entire dataset. Learning reliable embeddings for such infrequent tokens is statistically inefficient and prone to overfitting. By restricting the vocabulary to the most frequent tokens, the model receives more training examples per retained token, leading to better-estimated and more stable embeddings. In this sense, vocabulary reduction acts as a form of regularization by concentrating representation capacity on statistically meaningful patterns.

Similarly, always having multiple occurrences for each token, the model can learn richer and more reliable word representations. Rare tokens often contribute noise to the optimization process, as their gradients are based on limited evidence. Collapsing infrequent words into the `<UNK>` category reduces this variance and helps the model to focus on key underlying patterns in frequent words.

Vocabulary reduction can also affect optimization dynamics. Since softmax gradients are distributed over fewer competing words, the model may exhibit more stable loss trajectories, particularly under limited computational capacities.

All tokens outside the retained set were mapped to `<UNK>`. While this simplifies learning and reduces sparsity in the target distribution, it introduces ambiguity, as distinct rare words become indistinguishable. However, since 80% of types of tokens were mapped to `<UNK>`, it is important to ensure that the frequency of `<UNK>` does not increase to the point where it becomes a dominant signal, potentially distorting future generation quality of the deep learning model.

For this specific training dataset, after reducing the vocabulary to 20 percent of its original size, the `<UNK>` proportion in the training data was approximately 2.70%. This indicates that the retained vocabulary captured the vast majority of token occurrences, preserving most of the empirical probability mass while substantially reducing model complexity.

4.2 Training Dataset Construction

Training examples were constructed by treating each sentence as a sequence of next-token prediction tasks, using a fixed context window of 20 tokens chosen with reference to the median sentence length of 17 to capture most sentence-level dependencies. From a bias-variance perspective, this length strikes a balance. A longer window could reduce bias by incorporating more distant dependencies but would increase variance, memory usage, and training cost, while a shorter window would simplify learning at the cost of ignoring meaningful sentence-level structure. Left padding ensured consistent batching while preserving the most recent temporal information for the LSTM.

To control dataset size and training time, a stride-based downsampling mechanism was introduced. Rather than generating a training pair for every token position, only positions satisfying a stride condition were retained. While computationally efficient, this sampling strategy introduces important effects on token distribution which are discussed below.

4.2.1 The Stride and `<END>` Imbalance Problem

In the initial dataset construction loop, training pairs were generated at token positions i within each sentence. To reduce the number of training examples, a stride rule was applied: only positions where $i \bmod \text{stride} = 0$ were retained. However, the code also contained a special case intended to “always keep” sentence termination examples: if `keep_end` was enabled and the current target token was `<END>`, the stride filter was bypassed (i.e., the example was kept regardless of i).

The special `keep_end` condition was originally introduced to ensure that short sentences contributed meaningfully to training examples. Under pure stride-based subsampling, very short sentences might produce few or even no retained positions, leading to underrepresentation of sentence-final structure. The intention was therefore to guarantee that the model observed sufficient examples of sentence termination.

However, this asymmetry interacted with stride in an unintended way. While non-`<END>` targets were subject to stride-based downsampling, `<END>` targets were always retained. This strongly inflated frequency of `<END>` relative to other tokens. Although the underlying corpus distribution remained unchanged, the optimization objective seen by the model became biased toward sentence termination.

Thus, due to the skewed empirical target frequencies of tokens, the trained model assigned extremely high probability (approximately 0.90) to `<END>` at the first decoding step, causing immediate termination. Under the distorted training distribution, early termination became a statistically optimal strategy, leading to collapse in generation.

The issue was diagnosed by inspecting early-step generation probabilities, quantifying the empirical frequency of `<END>` in the training targets, and analyzing the distribution of retained training examples across sentence lengths.

These checks confirmed that the data construction process, rather than model capacity or optimization instability, was responsible for the collapse behavior.

4.2.2 Corrective Strategy

To restore distributional balance, stride was applied uniformly across all token positions, removing the asymmetric treatment of `<END>`. Then, to prevent the underrepresentation of short sentences and early sentence structure discussed above, the first and second token positions of each sentence were explicitly retained. This approach preserved sufficient coverage of sentence-initial contexts while still downsampling the data without distorting the empirical target distribution.

4.2.3 Interpretation

This highlights that sampling schemes directly shape the empirical target distribution, and even subtle asymmetries can induce severe generative distortions. Particularly in recursive language modelling, where the model internalizes token frequencies as probabilistic priors, any bias in class balance can translate directly into poor or completely collapsed generation behavior.

4.3 RNN Architecture Design

After experimentation with several architectural variants, the final recurrent language model was configured as:

$$\text{Embedding (128)} \rightarrow \text{LSTM (128)} \rightarrow \text{Dense (softmax over reduced vocabulary)}$$

The embedding layer maps discrete tokens into a 128-dimensional continuous space, enabling the model to learn distributed textual representations. A single-layer LSTM with 128 hidden units models sequential dependencies within the fixed 20-token context window. The final dense layer projects the LSTM hidden state directly onto the reduced vocabulary space via a softmax transformation, producing next-token probabilities. The model was trained using sparse categorical cross-entropy with the Adam optimizer and early stopping based on validation loss.

Several modifications were evaluated during development, including:

- Adding intermediate dense projection layers prior to the softmax.
- Reducing the number of LSTM hidden units to decrease training time.
- Increasing embedding dimensionality.
- Adjusting batch size and regularization strategies.

These alternatives were assessed under the practical constraint that training had to complete within approximately one hour.

Reducing the LSTM hidden size materially weakened sequence modelling capacity as reducing the hidden size to 72 units noticeably degraded validation accuracy, suggesting insufficient capacity to model longer-range dependencies within the 20-token window. This would be consistent with the established notion that in recurrent architectures, hidden state dimensionality is the primary bottleneck for contextual representation.

Adding an intermediate dense layer prior to the softmax increases computational cost disproportionately. Because the softmax layer scales with vocabulary size, any increase in the dimensionality of the preceding representation amplifies the cost of the most expensive operation in the network. Given the already substantial vocabulary even after reduction, this modification offered limited benefit relative to its computational cost.

Increasing embedding dimensionality beyond 128 yielded diminishing returns. While larger embeddings can improve representational expressiveness, the marginal gain in perplexity did not justify the additional parameter count and training time that went well beyond the given constraint.

A batch size of 128 provided a stable compromise between gradient noise and wall-clock efficiency. Smaller batches increased training time and variance in loss trajectories, while substantially larger batches did not materially improve convergence speed under the available constraints.

4.4 Generation Behaviour

Greedy decoding produces logically consistent but highly simplistic continuations, reflecting the model’s tendency to maximise local likelihood at each step. Introducing Top 10 sampling increases lexical diversity while broadly preserving grammatical structure, and overall yields more coherent sequences than the trigram baseline. However, the generated outputs remain heavily concentrated around generic review expressions such as “very good”, “great movie”, or “a little too long”. This indicates that the model has successfully learned dominant wording patterns from the training data but relies disproportionately on high frequency phrases, limiting its qualitative improvement from the trigram baseline.

While shorter sequences appear syntactically plausible and consistent, coherence deteriorates as generation length increases, particularly in longer samples where clause repetition emerges. This suggests that, despite the

Method	Generated Text
RNN Greedy	the film is a very good film
RNN Top-10 (Sample 1)	its a great movie but its not a great movie for the time of this film
RNN Top-10 (Sample 2)	the film was a little too long
RNN Top-10 (Sample 3)	its a good movie that i was not even surprised with it
RNN Top-10 (Sample 4)	and this isnt a bad film
RNN Top-10 (Sample 5)	i cant even consider how this film is about

Table 3: Comparison of RNN Greedy and Top-k Sampling Outputs

LSTM architecture, the model has limited capacity to maintain long range contextual dependencies. Overall, the RNN demonstrates improved fluency and diversity relative to the trigram model, but the outputs still lack the structural depth, specificity, and narrative consistency characteristic of authentic human written reviews.

4.5 Systematic Metric Comparison

Model Comparison

Model	Loss	Perplexity
Trigram (full vocab)	9.9943	21901.90
Trigram (reduced vocab)	8.3235	4119.68
RNN (reduced vocab)	5.2159	184.18

Table 4: Model Comparison by Loss and Perplexity

To compare the language models systematically, perplexity was used as the primary quantitative metric. Perplexity was computed for three models: the trigram model with the full vocabulary, the trigram model with the reduced vocabulary, and the RNN trained on the reduced vocabulary. In absolute terms, the RNN achieved substantially lower perplexity, indicating improved predictive performance over the n-gram baselines.

However, the evaluation procedures differ across models, which introduces an important limitation. The trigram perplexity reflects true sequential modelling over complete sentences, computed recursively across full token sequences. In contrast, the RNN perplexity was evaluated on sampled context target pairs, where the model predicts the next token given a fixed length context. Thus, the RNN perplexity reflects next-token prediction conditioned on the true preceding context and is not strictly comparable to fully recursive trigram perplexity.

For this reason, while perplexity values are directionally comparable and clearly indicate that the RNN captures more complex contextual dependencies than the trigram models, they should not be interpreted as perfectly aligned measures of generative quality.

5 Part 4: Deep Learning Classifier

While perplexity provides an intrinsic measure of next-token predictive performance, it does not directly assess whether generated sequences match the true data distribution. To evaluate distributional similarity more explicitly, a discriminative model was trained to distinguish between real and RNN-generated sentences.

5.1 Dataset Construction

A critical design decision was that real and generated sentences must be represented in the same token space. Since the generative RNN operates on the reduced vocabulary, real samples provided to the discriminator were also drawn from the reduced tokenized dataset. This prevents trivial separability based on vocabulary. If real sentences were represented in the full vocabulary while generated sentences contained many <UNK> tokens, the discriminator could use this discrepancy to identify fake reviews without learning meaningful structural differences.

Sentence length was standardized using percentile-based statistics from the corpus. Given a median sentence length of approximately 17 tokens and a 90th percentile near 34 tokens, a fixed maximum length of 30 was adopted as a pragmatic compromise. This choice captures large majority of sentence-level structure while maintaining computational efficiency and limiting excessive padding.

To ensure meaningful evaluation, the discriminator training set was balanced 50/50 between real and generated sentences. This prevents the classifier from achieving high accuracy by simply predicting the majority class and forces it to learn structural distinctions between the two distributions.

5.2 Discriminative Model Architecture

The discriminator architecture was intentionally simple:

Embedding $\rightarrow$ Gaussian Noise $\rightarrow$ GlobalAveragePooling $\rightarrow$ Dense (ReLU + Dropout) $\rightarrow$ Sigmoid

The embedding layer maps tokens into a continuous space shared across real and generated samples. Optional Gaussian noise was introduced as a regularization mechanism to reduce overfitting and discourage reliance on only surface-level token patterns. Global average pooling aggregates token embeddings into a fixed-length representation, effectively producing an embeddings summary. This design should make the model sensitive to token frequency patterns, stylistic consistency, and length effects.

A dense layer with ReLU activation and dropout provides nonlinear transformation and further regularization. The output layer uses a single sigmoid unit with binary cross-entropy loss. A two-class softmax would be equivalent but unnecessary in a binary classification setting. Lastly, early stopping based on validation loss was used to prevent overtraining.

5.3 Evaluation Metrics and Results

	Pred Fake	Pred Real
Actual Fake	396	11
Actual Real	53	340

Table 5: Confusion Matrix (Threshold = 0.5)

Metric	Value
Loss	0.3053
Accuracy	0.9200
ROC AUC	0.9751
PR AUC	0.9769
Precision (threshold = 0.5)	0.9687
Recall (threshold = 0.5)	0.8651
F1 Score (threshold = 0.5)	0.9140

Table 6: Discriminator Metrics

An AUC close to 1.0 indicates that the classifier can almost perfectly rank real sentences above generated ones. This implies that the generator has not fully captured the statistical structure and empirical distribution of real reviews. This suggests the RNN captures local grammatical structure but tends to produce overly regular text with reduced word diversity. These regularities become detectable by even a relatively simple classifier.

The recall for real sentences was approximately 86.5%, indicating that the discriminator is slightly conservative in predicting the real class. Exactly 53 genuine sentences were misclassified as generated, which suggests partial overlap between the two distributions: certain real sentences exhibit statistical patterns similar to generated text.

These findings are consistent with the architectural and training constraints of the generator, with a single-layer LSTM, reduced vocabulary, limited context window of 20 tokens, and relatively short training duration.

Importantly, the discriminator itself is intentionally shallow. Its strong performance indicates that even basic distributional features are sufficient to separate real and generated sentences. This reinforces the conclusion drawn from observed generation behaviour and perplexity analysis: while predictive performance improved relative to the trigram baseline, the RNN has not yet achieved distributional realism.

5.4 Integrating the Discriminator into GAN

A discriminative model trained to distinguish between real and synthetic reviews can be integrated into a generative framework in several ways to improve the quality of the synthetic text. This is because the discriminator provides a signal of authenticity that can be used to guide the generator towards better outputs.

One simple approach is rejection sampling. The RNN generator will produce candidate reviews, which will then be evaluated by the discriminator. Only samples that exceed a predefined authenticity threshold are retained (probability of being real). In this setup, the discriminator acts as a quality control mechanism, filtering out low quality or unrealistic generations. While this approach is simple to implement, such generative framework can become computationally inefficient if a large proportion of samples are rejected.

A more integrated approach is Generative Adversarial Network style framework, where generator and discriminator are trained side-by-side. In a GAN framework, the generator aims to produce reviews that the discriminator classifies as real. The discriminator is trained simultaneously to distinguish genuine reviews from synthetic ones. Over time, this interaction encourages the generator to produce increasingly realistic text, while discriminator becomes more proficient in distinguishing between real and generated text. Unlike image GANs, text generators produce discrete tokens, meaning the sampling operation is not differentiable. This prevents gradients from flowing directly from the discriminator through the generator. As a result, the generator is updated by increasing the probability of sequences that receive high realism scores from the discriminator, typically using reinforcement learning techniques.

A third approach is to use the discriminator as a reward function in a reinforcement learning fine tuning stage. After pretraining the RNN, the generator can be optimized to maximize a combined objective that includes both likelihood and discriminator score. In this setting, the discriminator's output represents a quantitative measure of how realistic a generated review appears.

Overall, incorporating a discriminative model into a generative framework adds a new consideration to the generation, which is no longer purely likelihood based modelling, but a process that considers some external metric of quality as guidance. For example, the discriminator provides an external signal that can reduce generic phrasing, encourage greater realism, and align synthetic reviews more closely with authentic human written content.

6 Part 5: General Reflection on the Use of LLMs

The most significant impact LLMs have had on my study and daily life is the increase in speed and convenience of interacting with vast amounts of data. Accessing information has become almost frictionless. When I have a complex or technical question, I no longer need to search across multiple sources and then synthesize the answer myself. In most cases, I receive a clear and structured response immediately, and if something remains unclear, I can refine my question and continue the dialogue. This has been particularly valuable for studying. If I struggle with a mathematical property, I can ask for an explanation or even a proof within seconds. If I encounter a coding issue, I can quickly identify where the error likely lies.

This significantly accelerates my learning process. I am able to move through material more efficiently and resolve misunderstandings immediately instead of being stuck on a problem for an extended periods of time. However, this efficiency comes with a trade-off. The ease of obtaining answers discourages deep thinking and prolonged engagement with difficult problems. There is a difference between understanding something because I have wrestled with it and understanding it because it has been clearly explained on demand. I sometimes question whether working through proofs and concepts more slowly and independently would lead to deeper internalization compared to reading concise, polished explanations generated instantly.

Another important advantage of using LLMs is the ability to focus on higher-value tasks. I can delegate mechanical work, such as grammar corrections or minor stylistic improvements to LLMs. Instead of spending time on repetitive and, to me, tedious editing, I can concentrate on more intellectually stimulating aspects of my work. In this sense, LLMs increase not just speed, but also the quality of how I allocate my attention.

At the same time, trust remains a central issue. Since these systems are non-deterministic, the question of how much to rely on their outputs is important to me. In my experience, they are correct in the overwhelming majority of everyday tasks, and mistakes occur relatively rarely. Therefore, in contexts where errors are not too costly, I find it reasonable to integrate AI outputs directly into my workflow. With that being said, an awareness of their probabilistic nature is in my view still necessary.

One limitation I have observed with LLMs is their tendency to be non-critical of their users. Even when configured to be critical, objective and direct through the personalization settings, they often agree with my ideas and rarely challenge my assumptions in a strong way. To me, this appears to be a natural consequence of their design. They generate the most probable continuation of a conversation rather than asserting an independent standpoint. As a result, they can reinforce my existing viewpoints instead of testing them rigorously.

Their strength lies in recognizing highly complex patterns over vast amounts of data and applying them in analytical or data-driven tasks. However, their limitations become more apparent in high-level decision-making contexts that involve unique trade-offs and far-reaching implications. From my experience, when analyzing strategic business scenarios, such as whether a company should fundamentally restructure its operations, an LLM tends to converge toward the direction implied by the user's framing.

For this reason, I believe the future role of professionals in analytics and business will increasingly center on making high-impact decisions in novel situations. These decisions cannot be determined solely by numerical outputs. They require combining analytics with contextual awareness, understanding broader implications, anticipating human reactions, and exercising judgment under uncertainty. In such environments, human insight remains essential, at least for now.